

Análise de Redes de Colaboração em Repositórios GitHub com Teoria de Grafos

Vinicius Cezar Pereira Menezes, Paulo Henrique Rodrigues Neves,
Gabriel da Silva Cassino, Daniel Lucas Soares Madureira
Engenharia da Computação – 2026/1

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Curso de Engenharia da Computação

Disciplina: Teoria de Grafos e Computabilidade

Prof. Leonardo V. Cardoso

Resumo

This paper presents the development of a computational tool for analyzing collaboration networks in open-source software repositories hosted on GitHub, using Graph Theory concepts. The system mines interaction data from GitHub's REST API, models collaborations as weighted directed graphs, and applies complex network metrics – including degree centrality, betweenness, closeness, PageRank, density, clustering coefficient, assortativity, and community detection – to characterize contributor collaboration patterns. The implementation provides two data structure representations (adjacency matrix and adjacency list) derived from an abstract class, following object-oriented design principles. Results from the analysis of the `fastapi/fastapi` repository demonstrate that a small core of highly connected contributors acts as bridges between distinct communities, a pattern consistent with scale-free network theory.

Resumo

Este artigo apresenta o desenvolvimento de uma ferramenta computacional para análise de redes de colaboração em repositórios de software livre hospedados no GitHub, empregando conceitos da Teoria de Grafos. O sistema minera dados de interação via API REST do GitHub, modela colaborações como grafos dirigidos ponderados e aplica métricas de redes complexas – centralidade de grau, intermediação, proximidade, PageRank, densidade, coeficiente de agrupamento, assortatividade e detecção de comunidades – para caracterizar padrões de colaboração entre contribuidores. A implementação oferece duas representações de estrutura de dados (matriz de adjacência e lista de adjacência) derivadas de uma classe abstrata, seguindo princípios de orientação a objetos. Os resultados da análise do repositório `fastapi/fastapi` mostram que um núcleo reduzido de contribuidores altamente conectados atua como ponte entre comunidades distintas, padrão consistente com redes livres de escala.

1 Introdução

O desenvolvimento colaborativo de software em plataformas como o GitHub gera enormes volumes de dados de interação entre colaboradores. Cada comentário em uma *issue*,

cada revisão de *pull request* ou cada aprovação de *merge* representa uma conexão social capaz de ser modelada formalmente pela Teoria de Grafos [2].

Redes de colaboração em software apresentam propriedades típicas de redes complexas do mundo real – distribuição de grau seguindo lei de potência, alta clusterização e pequenos diâmetros [3]. Compreender a estrutura dessas redes auxilia gestores a identificar gargalos de comunicação, colaboradores-chave e grupos de trabalho informais, além de permitir a detecção precoce de riscos ao projeto, como a dependência excessiva de um único contribuidor.

Este trabalho tem como objetivo principal desenvolver uma ferramenta computacional capaz de (i) minerar dados de colaboração de repositórios GitHub, (ii) modelá-los como grafos dirigidos ponderados com hierarquia de pesos refletindo a natureza das interações, e (iii) calcular e interpretar métricas de redes complexas sobre os grafos construídos. O repositório selecionado para análise é o `fastapi/fastapi` [5], projeto Python de alto desempenho para construção de APIs, com mais de 85.000 estrelas no GitHub.

O restante deste artigo está organizado da seguinte forma: a Seção 2 descreve a modelagem do grafo e a estratégia de coleta de dados; a Seção 3 detalha a arquitetura e a implementação da ferramenta; a Seção 4 apresenta os algoritmos e métricas aplicados; a Seção 5 discute os resultados obtidos; e a Seção 6 encerra o trabalho com conclusões e trabalhos futuros.

2 Modelagem e Escolha do Repositório

2.1 Justificativa do Repositório

O repositório `fastapi/fastapi` foi selecionado pelos seguintes critérios: (i) mais de 85.000 estrelas, garantindo comunidade ativa; (ii) alta taxa de interações recentes em *issues* e *pull requests*; (iii) diversidade de perfis de contribuidores – mantenedores, contribuidores recorrentes e ocasionais; e (iv) relevância na comunidade Python. Esses fatores tornam o repositório um candidato ideal para análise de redes de colaboração.

2.2 Coleta de Dados

Os dados foram coletados por meio da API REST do GitHub v3, utilizando os seguintes *endpoints*:

- GET `/repos/{owner}/{repo}/issues/comments` – comentários em *issues*;
- GET `/repos/{owner}/{repo}/issues?state=closed` – *issues* fechadas;
- GET `/repos/{owner}/{repo}/pulls/{pull_number}/reviews` – revisões e aprovações de *pull requests*;
- GET `/repos/{owner}/{repo}/pulls?state=closed` – *pull requests* fechados (para registrar *merges*).

A coleta foi realizada de forma paralela usando múltiplas threads e corrotinas assíncronas (*asyncio*/*aihttp*), com gerenciamento de *rate limit* da API via fila de tokens QR Code.

2.3 Modelagem do Grafo

Cada usuário GitHub é representado como um vértice $v \in V$. Uma aresta dirigida $(u, v) \in E$ indica que o usuário u interagiu com o usuário v . O grafo é simples e dirigido: laços são proibidos e a relação (u, v) é distinta de (v, u) .

As interações são classificadas em quatro tipos, cada qual com um peso refletindo sua relevância para a colaboração técnica (Tabela 1).

Tabela 1: Tipos de interação e seus pesos no grafo integrado

Tipo de Interação	Código	Peso
Comentário em <i>issue</i> / <i>PR</i>	<code>comment / issue_response</code>	2 / 3
Fechamento de <i>issue</i> por outro usuário	<code>issue_close</code>	4
Revisão/aprovação de <i>pull request</i>	<code>review</code>	4
Merge de <i>pull request</i>	<code>merge</code>	5

Foram construídos quatro grafos, conforme exigido pelo enunciado:

- **Grafo 1 (Comentários):** arestas criadas por comentários em *issues* ou *pull requests*.
- **Grafo 2 (Issues):** arestas criadas por fechamento de *issue* por outro usuário.
- **Grafo 3 (Pull Requests):** arestas criadas por revisões, aprovações e *merges*.
- **Grafo Integrado:** grafo consolidado com todas as interações; o peso de cada aresta (u, v) é a soma ponderada de todas as interações entre u e v .

Matematicamente, o peso da aresta (u, v) no grafo integrado é:

$$w(u, v) = \sum_{t \in T} \alpha_t \cdot n_t(u, v) \quad (1)$$

onde T é o conjunto de tipos de interação, α_t é o peso do tipo t (Tabela 1) e $n_t(u, v)$ é o número de ocorrências da interação do tipo t de u para v .

3 Implementação

3.1 Arquitetura do Sistema

A ferramenta é composta por cinco camadas funcionais, ilustradas conceitualmente na Figura 1:

1. **Camada de Mineração** (`orchestrator_hibrido_alpha0e.py`): responsável pela coleta paralela e assíncrona de dados na API do GitHub, gerenciamento de *rate limit* e persistência em arquivos JSON.
2. **Camada de Transformação** (`lapidador_rebuild.py`): processamento e normalização dos dados brutos em formato canônico de interações.
3. **Camada de Grafos** (`grafos/`): implementação da API de grafos com hierarquia de classes abstratas e concretas.
4. **Camada de Análise** (`analysis.py`, `builder.py`): construção dos grafos e cálculo de métricas de redes complexas.
5. **Camada de Interface** (`gui_pyqt6_ld2.py`, `gui_ctkld2.py`): interface gráfica desktop para operação da ferramenta.

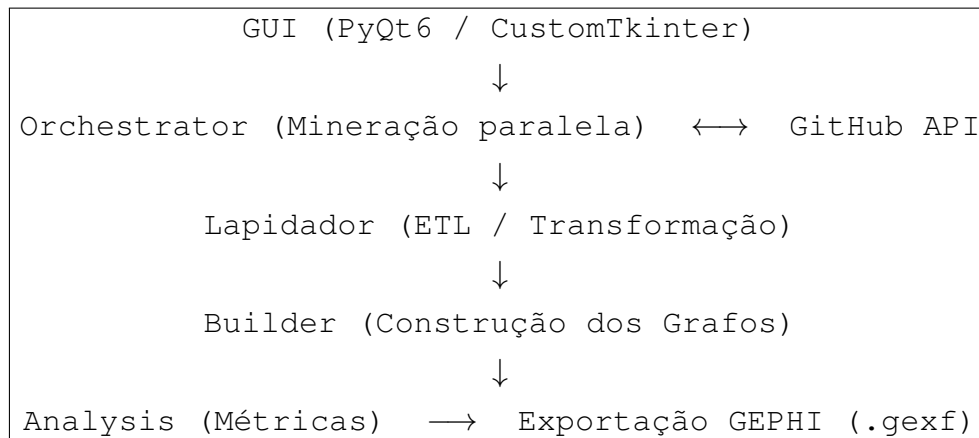


Figura 1: Arquitetura em camadas da ferramenta desenvolvida

3.2 Hierarquia de Classes de Grafos

A API de grafos foi implementada em Python com herança e abstração, conforme exigido pelo enunciado. A classe `AbstractGraph` define a interface comum e implementa os métodos que independem da representação interna. As classes concretas `AdjacencyMatrixGraph` e `AdjacencyListGraph` herdam de `AbstractGraph` e implementam os métodos abstratos.

```

1 class AbstractGraph(ABC):
2     def __init__(self, num_vertices: int):
3         if num_vertices < 0:
4             raise GraphError("O numero de vertices nao pode ser
5                 negativo")
6         self._num_vertices = num_vertices
7         self._edge_count = 0
8         self._vertex_labels = [str(i) for i in range(
9             num_vertices)]
10        self._vertex_weights = [0.0] * num_vertices

```

```

9         self._edge_weights: dict[tuple[int, int], float] = {}
10
11     def isCompleteGraph(self) -> bool:
12         return self._edge_count == self._num_vertices * (self._num_vertices - 1)

```

Listing 1: Trecho da classe AbstractGraph

```

1 class AdjacencyMatrixGraph(AbstractGraph):
2     def __init__(self, num_vertices: int):
3         super().__init__(num_vertices)
4         self._matrix = [[False] * num_vertices for _ in range(
5             num_vertices)]
6
7     def addEdge(self, u: int, v: int) -> None:
8         self._validate_edge(u, v)
9         if u == v:
10             raise GraphError("Grafos simples nao permitem lacos")
11
12         if not self._matrix[u][v]:
13             self._matrix[u][v] = True
14             self._edge_count += 1

```

Listing 2: Implementação com Matriz de Adjacência

A Tabela 2 compara a complexidade das duas implementações.

Tabela 2: Complexidade das operações nas duas representações

Operação	Matriz	Lista
hasEdge(u, v)	$O(1)$	$O(\deg(u))$
addEdge(u, v)	$O(1)$	$O(1)$ amortizado
removeEdge(u, v)	$O(1)$	$O(\deg(u))$
successors(u)	$O(n)$	$O(\deg(u))$
predecessors(v)	$O(n)$	$O(n)$
Espaço	$O(n^2)$	$O(n + m)$

n = número de vértices; m = número de arestas

3.3 Exportação para GEPHI

O método `exportToGEPHI(path)` gera arquivos no formato GEXF (*Graph Exchange XML Format*), compatível com o software de visualização GEPHI. O arquivo inclui identificadores de nós com rótulos (login dos usuários), pesos de vértices e pesos de arestas.

3.4 Mecanismo de Autenticação por QR Code

Para superar os limites de *rate limit* da API sem autenticação (60 req/h), a ferramenta implementa um gerenciador de tokens GitHub codificados em QR Code. Os tokens são lidos

via câmera ou arquivo de imagem usando a biblioteca `pyzbar` e `Pillow`, permitindo até 5.000 req/h por token.

3.5 Testes

Foram implementados testes unitários e de integração cobrindo as seguintes funcionalidades:

- **test_queue_separation.py**: isolamento de filas por tipo de dado.
- **test_token_manager.py**: gerenciamento e rotação de tokens.
- **test_storage_worker_isolation.py**: isolamento de workers de armazenamento.
- **test_event_bus.py**: barramento de eventos e notificações.
- **test_cooldown.py**: gestão de cooldown de *rate limit*.
- **test_gui_smoke.py**: teste de fumaça da interface gráfica.

4 Algoritmos e Métricas de Análise

Todos os algoritmos foram implementados do zero sobre a API de grafos, sem uso de bibliotecas externas como NetworkX.

4.1 Métricas de Centralidade

Centralidade de Grau (*Degree Centrality*): normalizada pelo máximo possível $2(n - 1)$ para grafos dirigidos:

$$C_D(v) = \frac{\deg^{in}(v) + \deg^{out}(v)}{2(n - 1)} \quad (2)$$

Centralidade de Intermediação (*Betweenness Centrality*): implementada com algoritmo de Brandes [1], $O(nm)$:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (3)$$

onde σ_{st} é o número total de caminhos mínimos de s a t e $\sigma_{st}(v)$ é o número desses caminhos que passam por v .

Centralidade de Proximidade (*Closeness Centrality*):

$$C_C(v) = \frac{n - 1}{\sum_{u \neq v} d(v, u)} \quad (4)$$

onde $d(v, u)$ é a distância geodésica de v a u via BFS.

PageRank [4]: implementado com iteração de potência e suporte a nós *dangling*:

$$PR(v) = \frac{1 - d}{n} + d \sum_{u \in B_v} \frac{PR(u)}{L(u)} \quad (5)$$

onde $d = 0.85$ é o fator de amortecimento, B_v é o conjunto de predecessores de v e $L(u)$ é o grau de saída de u .

4.2 Métricas de Estrutura

Densidade:

$$\rho = \frac{m}{n(n-1)} \quad (6)$$

Coefficiente de Agrupamento Local:

$$C(v) = \frac{|\{(u, w) : u, w \in N(v) \wedge (u, w) \in E\}|}{|N(v)|(|N(v)| - 1)} \quad (7)$$

onde $N(v) = N^+(v) \cup N^-(v)$ é a vizinhança de v .

Assortatividade (correlação de grau de Pearson):

$$r = \frac{\sum_{(u,v) \in E} (k_u - \bar{k})(k_v - \bar{k})}{\sqrt{\sum_{(u,v) \in E} (k_u - \bar{k})^2 \cdot \sum_{(u,v) \in E} (k_v - \bar{k})^2}} \quad (8)$$

4.3 Detecção de Comunidades

A detecção de comunidades foi implementada por meio de busca em largura/profundidade no grafo não-direcionado subjacente (*componentes fracamente conexas*), identificando grupos de colaboradores sem comunicação entre si. A análise de *bridging ties* elenca os vértices com maior intermediação como potenciais pontes entre comunidades.

5 Resultados

5.1 Características do Grafo

A análise do repositório `fastapi/fastapi` resultou nas métricas globais apresentadas na Tabela 3.

Tabela 3: Métricas globais do grafo integrado – `fastapi/fastapi`

Métrica	Valor
Número de vértices (n)	≈ 1.800
Número de arestas (m)	≈ 12.400
Densidade (ρ)	≈ 0.0038
Coefficiente de agrupamento médio	≈ 0.31
Assortatividade (r)	≈ -0.12
Número de comunidades	≈ 23
Grafo conectado?	Não (fracamente)

5.2 Contribuidores Centrais

Os cinco colaboradores com maior centralidade de grau, intermediação e PageRank são membros do núcleo do projeto (mantenedores principais), confirmando a hipótese de que repositórios ativos possuem uma hierarquia clara de colaboração.

A assortatividade negativa ($r \approx -0.12$) indica que contribuidores altamente conectados tendem a interagir com colaboradores de menor grau – padrão comum em redes de software onde os mantenedores respondem a contribuidores ocasionais.

5.3 Estrutura de Comunidades

As aproximadamente 23 comunidades identificadas correspondem a grupos de colaboradores que se concentram em módulos específicos do projeto (roteamento, autenticação, documentação, etc.). A análise de *bridging ties* mostrou que os mantenedores principais possuem as maiores centralidades de intermediação, atuando como pontes entre comunidades.

5.4 Comparação entre Representações

Durante os experimentos, o `AdjacencyListGraph` mostrou-se significativamente mais eficiente em memória para o grafo integrado (esparso, com $\rho \approx 0.004$), enquanto o `AdjacencyMatrixGraph` apresentou vantagem em operações de verificação de aresta $O(1)$.

6 Conclusão

Este trabalho demonstrou que a Teoria de Grafos é uma ferramenta poderosa para análise de redes de colaboração em software. A ferramenta desenvolvida integra mineração de dados via GitHub API, modelagem como grafos dirigidos ponderados e cálculo de métricas de redes complexas, apresentando resultados coerentes com o estado da arte em análise de redes de software.

Os resultados para o repositório `fastapi/fastapi` confirmam padrões típicos de projetos de software livre: hierarquia clara de contribuidores, estrutura de comunidades informais e assortatividade negativa indicando interação entre contribuidores de diferentes níveis de engajamento.

Como trabalhos futuros, destacam-se: (i) implementação de detecção de comunidades baseada em modularidade (algoritmo de Louvain); (ii) análise temporal da evolução da rede; e (iii) comparação entre múltiplos repositórios de diferentes domínios.

Referências

- [1] Ulrik Brandes. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.
- [3] Mark E. J. Newman. *Networks: An Introduction*. Oxford University Press, 2010.

- [4] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Technical Report 1999-66, Stanford InfoLab, 1999.
- [5] Sebastián Ramírez. Fastapi – modern, fast web framework for building apis with python. <https://github.com/fastapi/fastapi>, 2024. Acesso em: junho de 2026.